STRIPE PROJECT

1. OLTP Data System

Annexes

1.D. OLTP Replication, Failover and Disaster Recovery Strategy

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / OLTP Data System / Annexe D / v36 -- 2026/11/05.*

1.D.1. Contexte

Ce document annexe présente les stratégies de réplication, de basculement (*failover*) et de reprise après sinistre (*disaster recovery*) du système *OLTP*. Bien que ces fonctions ne soient pas exigées dans les livrables du projet [D0], elles sont mentionnés plus ou moins explicitement dans les exigences :

- La réplication continue des données critiques est évoquée implicitement dans l'exigence d'affaire *BR1* (*"[...] real-time data synchronization"*). Elle implique une exigence de haute disponibilité en écriture qui justifie une procédure de basculement spécifique (basculement entre le site principal et l'une de ses répliques en cas de panne du site principal).
- La procédure de reprise après sinistre est exigée explicitement dans *BR1* (*"[...] disaster recovery"*).

Technologie

- Le *SGBD OLTP* est supporté par *PostgreSQL* (16.15).

Limites

- Haute disponibilité *locale* vs *globale* :
  - L'exigence technique *TR3* de [D0] invoque une haute disponibilité *globale* et une capacité à monter en charge *horizontalement* pour tous les systèmes de données du cas d'étude (incluant, donc, le système *OLTP*).
  - La topologie proposée dans ce document assure une haute disponibilité des données critiques (en écriture et en lecture) mais seulement dans la mesure où la charge en écriture peut être supportée par *un seul cluster* (1 seul primaire en écriture, 1 seul cluster).
  - Pour absorber la charge d'écriture *horizontalement*, une stratégie de *sharding* est nécessaire (le *sharding* est une technique d'accompagnement de la montée en charge basée sur un partitionnement *horizontal*). Chaque *shard* (*~partition*) implique un cluster dédié qui reprend à son compte la topologie mono-cluster présentée dans cet annexe.
- Basculement *inter-région* (*Disaster Recovery*):
  - La procédure de basculement inter-région en cas de désastre régional est évoquée dans la section *Failover* (§1.D.3) et intégré dans l'architecture de réplication (§1.D.6) mais aucune solution technique n'est proposée.

1.D.2. Réplication

1.D.2.1. Principes

Le système *OLTP* supporte l'écriture de données de criticité différentes :

- Les données d'*enjeu critique*, financier ou contractuel :
  - `Transaction` ,
  - `Subscription` ,
  - `Refund` / `Chargeback` ,
  - `FraudScore` / `FraudDecision` ;
- Les journaux de sécurité et conformité :
  - `DataAccessLog` / `ChangeAuditLog` ,
  - `SecurityIncident` ,
  - `ConsentRecord` ,
  - `DataSubjectRequest` ,
  - `FraudScoringLog` / `FraudReview` ;
- Les liste de références :
  - `Country` , `Currency, statuts, etc.

Chacune de ces familles des données a des exigences de continuité propres :

Criticité	RPO (Recovery Point Objective)	RTO (Recovery Time Objective)
Enjeu critique	~0 (aucune transaction confirmée ne doit être perdue)	Quelques secondes à 1 minute
Journaux	Jusqu'à ~1 minute (écriture déjà asynchrone par conception)	Quelques minutes
Référentiels	Quelques minutes	Quelques minutes (lecture seule tolérable en repli)

L'enjeu du système de réplication physique est d'assurer la *continuité de service* en cas de panne d'un serveur en minimisant la perte des données critiques (*RPO~0*) et la durée d'interruption des services (*RTO<<1min*).

La procédure de basculement d'une base primaire à une base répliquée à l'intérieur d'une même région (*Failover*) fait l'objet de la section §1.D.3. Le cas d'une reprise de service à la suite d'une panne couvrant une région entière (*Disaster Recovery*) est présenté en section §1.D.4.

1.D.2.2. Réplication physique en streaming

La réplication physique en flux sous *PostgreSQL* est assurée par le mécanisme natif de "*streaming replication*". Au moins trois répliques sont mises en place :

- 1 réplique *synchrone* et 1, ou plus, réplique(s) de relève de la synchronisation, *dans la même région* (zone de disponibilité) :
  - Le commit d'une transaction SQL sur `Transaction` n'est confirmé au client qu'après avoir été répliqué sur le noeud de réplique synchrone ;
  - Ce mécanisme assure un *RPO* négligeable sur l'écriture des données d'enjeu critique ;
- 1 ou plusieurs répliques *asynchrones* supplémentaires, dans une région géographique distincte :
  - L'objectif est d'absorber la perte d'une région entière (dans le cadre d'une reprise après sinistre) sans pénaliser la latence d'écriture des données d'enjeu critique.

1.D.3. Bascule (failover)

1.D.3.1. Principes

Dans un écosystème de réplication, il y a plusieurs plusieurs copies de la base :

- une *primaire* (la seule base seul qui accepte les écritures),
- des *répliques* (qui reçoivent en continu les états de la copie primaire).

Le but du processus de *bascule* (*failover*) est de 1) promouvoir une *réplique* en nouveau *primaire* en cas de panne du *primaire* courant, et de 2) rediriger les applications vers elle, le plus vite possible et sans perdre de transactions.

Le *failover* est un processus en quatre étapes :

1. *Détection* : constater/confirmer que le primaire est en panne ;
2. *Élection* : choisir la réplique qui remplace le primaire en panne ;
3. *Promotion* : transformer techniquement cette réplique en primaire ;
4. *Redirection* : faire en sorte que (1) les applications se connectent au nouveau primaire, (2) les anciennes répliques suivent le nouveau.

Limites

Monitoring. La proposition courante n'intègre pas la supervision des pannes ni des états du processus de *failover*.

1.D.3.2. Fonctions *PostgreSQL*

PostgreSQL fournit les briques de réplication et la commande de promotion, mais aucun mécanisme pour décider qui, et quand, promouvoir ni pour rediriger les applications. On voit dans le tableau ci-dessous que la plupart des étapes du *failover* (cf. §1.D.3.1) ne sont pas couvertes :

#	Étape	Natif PostgreSQL
-	Réplication en continu du primaire vers les répliques (streaming replication)	Oui
-	Réplication synchrone (commit confirmé après l'actualisation de la réplique)	Oui
1.	Détection de la panne du primaire	Non
2.	Élection automatique de la meilleure réplique	Non
3.	Promotion d'une réplique en primaire	Oui (manuelle avec `pg_promote`)
4a.	Redirection des applications	Non
4b.	Remise en ordre des autres répliques et de l'ancien primaire	Non (outils existants (`pg_rewind`, `pg_basebackup`) mais orchestration manquante)

1.D.3.3. Architecture retenue

La détection, la confirmation de la panne, l'élection du nouveau primaire, la redirection des applications et l'orchestration des répliques reposent sur une source de vérité unique et fiable : un magasin de configuration distribué (*DCS*, *Distributed Configuration Store*) qui permet d'identifier le noeud primaire de manière certaine. Sans cette source unique et fiable, deux nœuds pourraient se croire primaires en même temps (*split-brain*) et accepter des écritures divergentes.

Quatre composants se partagent le travail de failover :

Composant	Rôle	Étapes *failover* couvertes
etcd	*DCS* : héberge le verrou de leader et l'état partagé du cluster	Source de vérité (1, 2)
Patroni	1 agent / nœud : pilote *PostgreSQL* en fonction de l'état d'*etcd*	Détection, élection, promotion, remise en ordre (1, 2, 3, 4b)
HAProxy	Point d'entrée unique, aiguille l'écriture vers le primaire	Redirection (4a)
PgBouncer	Mutualise les connexions clientes	Hors *failover* : protège *PostgreSQL*

Le diagramme §1.D.6 illustre l'architecture proposée.

> Noter que *PgBouncer* est installé sur chaque nœud *PostgreSQL* . *HAProxy* contrôle la santé des nœuds par l'API REST de *Patroni* mais envoie le trafic vers *PgBouncer*.

Objectif de service : la bascule automatique doit rétablir l'écriture en moins d'une minute, si possible en quelques secondes, conformément au *RTO* des données critiques §1.D.2.1. La tenue de cet objectif dépend de la configuration des temporisations.

1.D.3.4. Détection, élection et promotion : *Patroni* + *etcd*

etcd

etcd est un magasin clé-valeur, open-source, distribué, fortement cohérent (consensus *Raft*). Son rôle est d'héberger les informations suivantes :

- *Le verrou de leader* : une clé du type « le primaire actuel est le nœud A », que le primaire doit renouveler régulièrement (un bail, avec une durée de vie de 20 à 30 s).
- *L'état partagé* : configuration dynamique du cluster, liste des membres, etc.

Dimensionnement : un cluster *etcd* de 3 membres répartis sur 3 zones de disponibilité de la région principale (quorum = 2, tolère la perte d'un membre). *etcd* n'est pas étendu à la région de reprise après sinistre : la latence inter-région est trop variable pour fiabiliser un quorum (cf. §1.D.3.9).

Patroni

Patroni est un outil open source qui permet d'orchestrer la première partie de la procedure de failover (détection, élection, promotion). Un agent *Patroni* tourne à côté de chaque instance PostgreSQL et la pilote :

- *Sur le primaire* :
  - renouvelle en permanence le verrou de leader.
  - auto-protection : rétrograde le primaire s'il n'arrive plus à joindre *etcd*. C'est la parade au *split-brain* : un primaire isolé sait qu'il a peut-être perdu son statut et se met en sécurité.
- *Sur les répliques* :
  - surveille le verrou. S'il expire (le primaire ne répond plus), les répliques candidates entrent en compétition ; celle qui obtient le verrou gagne (avec un critère de fraîcheur : on n'élit pas une réplique en retard sur les données).
  - réaligne les autres répliques sur le nouveau primaire,
  - réintégre éventuellement l'ancien primaire comme réplique.

Temporisations

Trois paramètres de la configuration dynamique de *Patroni* (stockée dans *etcd*, commune à tous les nœuds) fixent la vitesse de détection :

Paramètre	Rôle	Défaut	Minimum
`ttl`	Durée de vie du verrou de leader : le délai avant qu'une panne soit constatée	30 s	20 s
`loop_wait`	Période de la boucle de chaque agent (renouvellement, surveillance)	10 s	1 s
`retry_timeout`	Délai accordé aux opérations *etcd* / *PostgreSQL* avant échec	10 s	3 s

Réplication synchrone et critère d'élection

En mode synchrone (`synchronous_mode`), *Patroni* s'assure que seule la réplique synchrone (ou l'une d'elles) peut être promue automatiquement, ce qui garantit qu'aucune transaction confirmée au client n'est perdue (*RPO* ≈ 0).

Lorsque plus aucune réplique synchrone n'est disponible, le *mode strict* (`synchronous_mode_strict`) transforme la perte de la dernière réplique synchrone en indisponibilité d'écriture (le primaire bloque les écritures plutôt que de confirmer des transactions non répliquées). Il est donc recommandé de disposer, dans la région principale, de deux répliques synchronisées : la perte d'un seul nœud ne bloque alors jamais les écritures et ne prive jamais le cluster d'un candidat à la promotion.

> Le mode *strict* est le seul mode de fonctionnement de *Patroni* qui permet de garantir que *RPO ≈ 0*.

Les répliques asynchrones de la région de reprise après sinistre sont exclues de l'élection automatique par les *tags Patroni* `nofailover` (jamais promue automatiquement) et `nosync` (jamais choisie comme réplique synchrone).

Protections contre le *split-brain*

Un *split-brain* est une situation dans laquelle deux nœuds d'un cluster se croient primaires en même temps et acceptent chacun des écritures.

Il existe plusieurs niveaux de protection complémentaires :

1. *Le verrou de leader dans etcd :* un seul nœud peut détenir le verrou à un instant donné. Comme *etcd* fonctionne par quorum (majorité), la partie minoritaire d'une partition réseau ne peut ni obtenir ni renouveler le verrou.
2. *Rétrogradation sur perte du verrou*. Si le primaire ne parvient pas à renouveler son verrou, il passe de lui-même en lecture seule, avant qu'une autre réplique soit promue.
3. *Protection contre le blocage de Patroni* (*watchdog*). Si le processus *Patroni* se fige alors que *PostgreSQL* continue de tourner, il ne renouvelle plus le verrou et ne peut pas se rétrograder lui-même : l'ancien primaire reste actif pendant qu'une autre réplique est promue. Un *watchdog* matériel ou logiciel (`/dev/watchdog`) force alors le redémarrage du nœud.
4. *HAProxy* ajoute une dernière barrière côté applications : `on-marked-down shutdown-sessions` coupe les connexions vers l'ancien primaire, et seul le nœud qui répond « primaire » à *Patroni* reçoit du trafic.

Lors d'une rétrogradation du primaire sur perte du verrou, une indisponibilité éventuelle d'*etcd* (perte du quorum) implique un arrêt des écritures. Avec l'activation du mode `failsafe_mode`, une panne d'*etcd* seule ne coupe donc pas l'écriture car ce mode permet au primaire de rester actif tant qu'il peut joindre tous les membres connus du cluster.

Remise en ordre et sauvegardes

Après une bascule, l'ancien primaire peut contenir des transactions que le nouveau primaire n'a jamais reçues : il ne peut pas redevenir réplique tel quel. *Patroni* utilise `pg_rewind` pour le réaligner sans copie complète. Si le *rewind* échoue ou n'est pas possible, le nœud

est reconstruit depuis une sauvegarde *pgBackRest* (cf. §1.D.5).

1.D.3.5. Redirection des applications : proxy de connexion

Pour permettre à la procédure de *failover* de rediriger les applications vers le nouveau primaire sans qu'elles s'en aperçoivent, un proxy de connexion doit être inséré devant le cluster, de telle sorte que les applications ne référencent jamais directement un noeud physique.

Le rôle du proxy de connexion est, d'une part, d'exposer une adresse unique aux applications, d'autre part, d'interroger régulièrement l'API REST *Patroni* de chaque nœud pour identifier le primaire courant. Le proxy renvoie le trafic d'écriture au nœud primaire, et seulement au noeud primaire. Après une bascule, le nouveau primaire s'identifie, l'ancien ne répond plus, et le proxy redirige tout seul le trafic en quelques secondes.

Solutions proposées : *HAProxy*, ou un service managé (*RDS Proxy*, *Cloud SQL Auth Proxy*, etc.).

HAProxy est un répartiteur de charge et un proxy réseau générique, open-source, et très répandu. Il reçoit des connexions sur une adresse fixe et les redirige vers un ou plusieurs serveurs en aval, en vérifiant régulièrement lesquels sont en bonne santé.

HAProxy travaille dans la couche de *transport réseau (TCP)* : il ne comprend pas le protocole *PostgreSQL* et transfère simplement les octets vers le bon serveur.

1.D.3.6. Pooling de connexions

Un déploiement du système *OLTP* à grande échelle implique la capacité à gérer des milliers de connexions clientes concurrentes en chaque noeud. Dans le cas d'un système basé sur *PostgreSQL*, cette exigence doit être couverte par une couche logicielle externe, dite de *pooling de connexions*, car *PostgreSQL* n'a pas nativement la capacité à absorber autant de connexions simultanées.

> *PostgreSQL* gère les connexions clientes concurrentes en dédiant un *processus système* à chaque connexion. Etant donné le coût d'un *processus système* en terme de ressource mémoire/CPU (comparé à un thread léger, par exemple), le serveur *PostgreSQL* s'écroulerait bien avant d'avoir absorber un millier de connexions (le nombre maximum de connexions que le serveur accepte de traiter simultanément est de 100 par défaut).

Solutions de *pooling*

Solutions proposées : *PgBouncer*, ou un service managé (généralement, le même que celui qui gère déjà le proxy de connexion : *RDS Proxy*, *Cloud SQL Auth Proxy*, etc.).

PgBouncer est un *pooler de connexions PostgreSQL*, open source, très léger. Il permet de gérer des milliers de connexions clientes concurrentes sans multiplier les processus *PostgreSQL* en exploitant le fait que seul un petit nombre de ces connexions sont actives au meme moment et en ne présentant à *PostgreSQL* qu'une dizaines de connexions réelles.

1.D.3.7. Fenêtre de bascule

La durée d'une bascule est la période pendant laquelle le service d'écriture est interrompu : de l'instant où le primaire cesse de répondre jusqu'à celui où les applications peuvent de nouveau écrire sur le nouveau primaire. Elle se décompose, pour la perte brutale d'un nœud primaire, de la façon suivante :

- Expiration du verrou de leader -> 20 à 30 s ,
- Prise en compte par les répliques puis élection -> 1 à 10 s,
- Promotion -> ~ 1 à 3 s,
- Redirection par *HAProxy* -> ~ 6 s.

> Fenêtre de bascule :

- Meilleur cas -> ≈ 28 s,
- Pire cas -> ≈ 49 s,
- Ordre de grandeur moyen -> ≈ 39 s.

Cohérence avec le RTO : la fenêtre cible de 30 s est cohérente avec la borne haute du *RTO* des données critiques (« jusqu'à 1 minute », §1.D.2.1) mais pas avec sa borne basse (« quelques secondes », §1.D.2.1). La formulation du *RTO* en §1.D.2.1 devra être harmonisée avec cette fenêtre.

1.D.3.8. Supervision et alertes

Une bascule peut avoir lieu sans que personne ne le sache. Ce document ne spécifie pas la chaîne de supervision complète (collecte, tableaux de bord, astreinte), qui relève de l'exploitation, et se limite à l'identification des évènements à surveiller :

Evènement	Source	Condition d'alerte
Changement de primaire	API REST et métriques de *Patroni*, journaux	alerte : plusieurs bascules en peu de temps
Santé du cluster *etcd*	Métriques d'*etcd*	Perte d'un membre (le quorum n'a plus de marge) ; élections de leader fréquentes
Réplique synchrone disponible	Statistiques de réplication de *PostgreSQL*, état de *Patroni*	Aucune réplique synchrone (écritures bloquées en mode strict)
Retard de réplication	Statistiques de réplication de *PostgreSQL*	Retard croissant, notamment vers la région de reprise après sinistre
Nœuds sortis du pool *HAProxy*	Statistiques d'*HAProxy*	Aucun nœud primaire disponible
Connexions saturées	Statistiques de *PgBouncer*	Clients en attente de connexion serveur

1.D.3.9. Répliques asynchrones inter-régions

Les répliques *asynchrones* inter-régions ne participent pas à l'élection automatique (car la latence de réseau inter-régions est trop variable pour fiabiliser un quorum) : elles portent les *tags* `nofailover` et `nosync`, et le cluster *etcd* n'est pas étendu à leur région. Elles servent au recouvrement après sinistre (§1.D.4).

1.D.4. Reprise après sinistre (*disaster recovery*)

1.D.4.1. Failover (A) vs Disaster Recovery (B)

	Failover (région A)	**Disaster Recovery (région B)**
Déclencheur	Panne d'un nœud ou d'une zone	Perte de la région entière
Bascule	Automatique (*Patroni*)	Manuelle (décidée par un humain)
Réplique concernée	Synchrone	Asynchrone
Perte de données possible	Aucune (RPO ≈ 0)	Quelques secondes (RPO >> 0)

> Les répliques asynchrones montrent toujours un léger retard de synchronisation par nature. La promotion d'une réplique asynchrone s'accompagne donc toujours d'une perte probable des dernières transactions non encore répliquées (RPO >> 0).

1.D.4.2. Bascule inter-région

En cas de *perte totale de la région primaire*, la bascule inter-région est déclenchée *manuellement* vers la réplique asynchrone distante, plutôt qu'automatisé, pour éviter un failover inter-région intempestif sur un simple pic de latence réseau. Le processus est documenté et testé régulièrement (exercice de bascule programmé, a minima trimestriel).

1.D.4.3. Signalisation de la perte des tables des journaux après bascule

Toute purge ou incohérence détectée sur `DataAccessLog` / `ChangeAuditLog` après un incident de bascule doit elle-même être tracée dans `SecurityIncident`, la table conçue pour capturer les incidents est aussi celle qui documente les incidents touchant l'infrastructure elle-même.

1.D.5. Sauvegarde vs. Réplication

Comme le système de réplication, le système des sauvegardes et des restaurations est capable de restaurer les données perdues en minimisant la perte des données (*RPO*) et, si possible, la durée d'interruption des services (*RTO*). Le point de differentiation le plus important est qu'il échange une contrainte de continuité de service contre une contrainte d'historisation des données : en principe, il est capable de restaurer les données dans n'importe quel état *passé*. Typiquement il doit être capable de restaurer les données saines les plus récentes en cas de rançonnage ou de corruption des données (ce qu'un système de réplication ne peut pas faire puisque, par définition, il réplique les données corrompues).

Basiques d'un système de sauvegarde et de restauration

Un système complet de sauvegarde et de restauration en entreprise en défini par :

- Un *plan de recouvrement* définissant la gestion des accès, la gestion des secrets, la documentation, la supervision et la maintenance des infrastructures réseau & stockage, et les tests de restauration automatisés.
- Une infrastructure comprenant des serveurs de stockage *indépendants*, des processus de compression, de chiffrage, de nettoyage automatique, et, si possible, de stockage *immuable*.
- Un système de gestion des sauvegardes et des restaurations :
  - Recevoir & archiver les segments *WAL* ,
    - *WAL* = Write-Ahead-Log *(le système de journalisation à la base du mécanisme de* CDC *sous* PostgreSQL*) ;
  - Réaliser les sauvegardes de base (complètes, différentielles, incrémentales) ;
  - Purger les archives en fonction de la politique de *rétention* ;
  - Vérifier l'intégrité des sauvegardes (sommes de contrôle), compresser, chiffrer ;
  - Envoyer les sauvegardes vers différentes cibles (disque, S3, stockage objet cloud) ;
  - Orchestrer la restauration.

Principes de la sauvegarde continue

La *sauvegarde continue* est une procédure de sauvegarde avancée qui permet de restaurer la base en n'importe quel point passé d'une fenêtre de retention donnée.

Principes :

- Sauvegarde :
  - Sauvegarde continue des états sur $[T_1, T_2]$:
    1. faire une sauvegarde complète de la base à l'instant T_1 ;
    2. archiver continûement le *journal des transactions confirmées* sur la période $(T_1, T_2]$ (où $\Delta = T_2 - T_1$ est la *fenêtre de retention*).

- Restauration :
  - Restauration *PITR* (*Point In-Time Recovery*) en $T_3 \in (T_1, T_2]$:
    1. Restaurer la base dans son état en T_1 à partir de la sauvegarde complète ;
    2. Rejouer toutes les modifications enregistrées dans le *journal des transactions confirmées* jusqu'en T_3.

Solutions de sauvegarde sous PostgreSQL

Les fonctions précédentes de sauvegarde continue sont intégrées nativement dans *PostgreSQL* :
- snapshot ponctuel de la base,
- restauration d'un sauvegarde ponctuelle,
- archivage *continu* du *WAL* (qui joue le rôle ici du transactions confirmées),
- rejeu du *WAL* archivé.

La gestion des sauvegardes et des restaurations sous *PostgreSQL* peut être assurée par *pgBackRest* ou par un équivalent managé.

pgBackRest est une extension open source de *PostgreSQL* qui se charge des points suivants :

- recevoir & archiver les segments *WAL* ;
- réaliser les sauvegardes de base (complètes, différentielles, incrémentales) ;
- purger en fonction de la politique de rétention ;
- vérifier l'intégrité des sauvegardes (sommes de contrôle), compresser, chiffrer ;
- envoyer les sauvegardes vers différentes cibles (disque, S3, stockage objet cloud) ;
- orchestrer la restauration.

Fenêtres de rétention

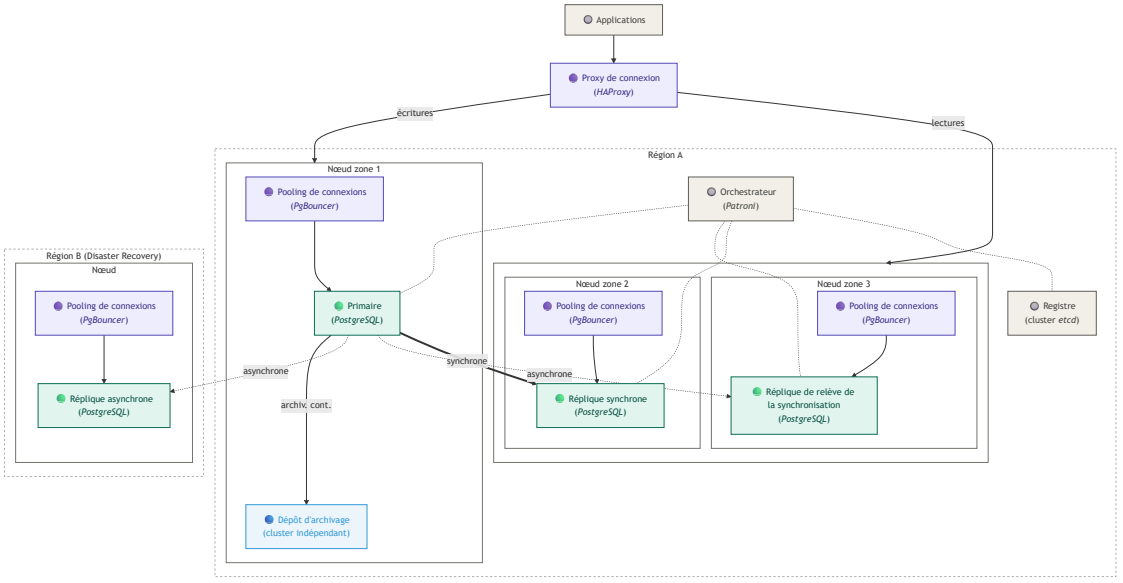
La fenêtre de rétention recommandée pour les sauvegardes continues : 35 jours, en accord avec les exigences de délai RGPD/PCI-DSS.

Dimensionnement de l'archivage

Snapshots complets hebdomadaires + 35 j -> 5 à 6 sauvegardes simultanées

1.D.6. Architecture du système de réplication

Le diagramme suivant récapitule l'architecture de la solution technique de réplication, bascule et recouvrement présentée en §1.D.2, §1.D.3 et §1.D.4.



> *Légende* :
>
> - 🔘 : *les applications et le plan de contrôle (Patroni, etcd)*
> - 🟢 : *les instances PostgreSQL (primaire et répliques)*
> - 🟣 : *les proxys (HAProxy et PgBouncer)*
> - 🔵 : *les dépôts d'archivage*
> - `---->` : Trafic applicatif
> - `-.->` : Supervision (bail, état)
> - `===>` : Réplication Synchrone

- `-.->` : Réplication Asynchrone

Références

Annexes

- [D2-B] OLTP Physical Model

Spécifications

- [D0] Stripe Project Introduction